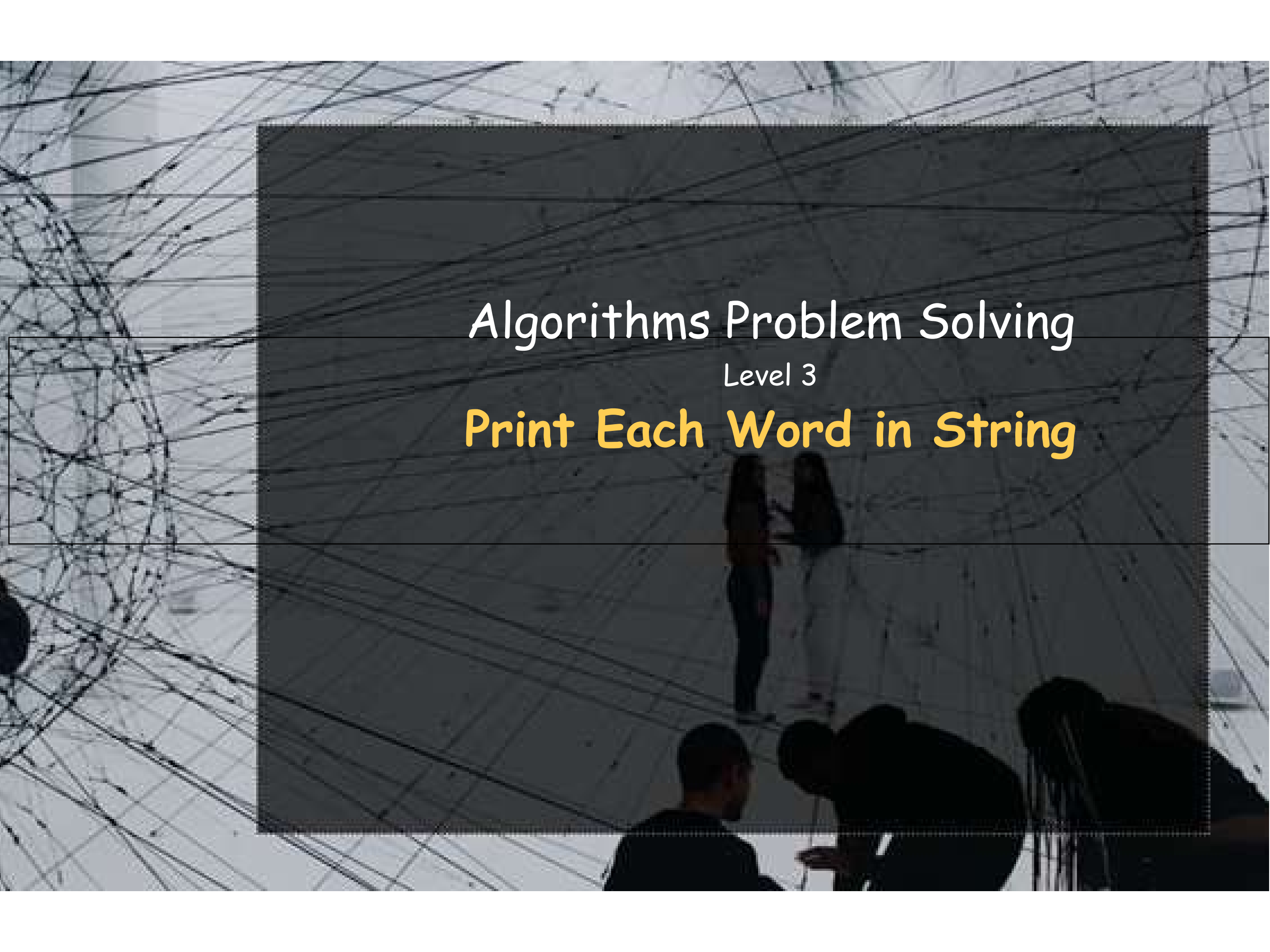


# Algorithms Problem Solving

## Level 3

### Print Each Word in String

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the lower part of the image, the silhouettes of several people are visible, some standing and some sitting, suggesting a collaborative workspace or a public area. The overall color palette is muted, with greys, blues, and the warm tones of the light coming through the glass.

# Algorithms Problem Solving

Level 3

## Print Each Word in String

# Definition

## Manual String Tokenization

 Program separates a sentence into individual words and prints each word on a separate line.

 It demonstrates manual tokenization using ``find()`, `substr()`, and `erase()`.`

# Explanation

## Delimiter Search

-  ``str.find(" ")`` locates the next space separating two words.
-  The returned position identifies where the current word ends.

## Substring Extraction

-  ``str.substr(0, pos)`` extracts the characters before the delimiter.
-  The extracted substring represents one word.

## Source String Modification

-  After processing a word, ``str.erase(0, pos + 1)`` removes it and its delimiter.
-  The remaining string is processed again.

## Final Word Handling

-  When no delimiter remains, the remaining string is the final word.
-  That last word must also be printed.

# Procedures

1. Read the complete sentence using ``getline()'`.
2. Search for the next space.
3. Extract the word before the space.
4. Print the word.
5. Erase the processed portion.
6. Continue until the remaining string contains the final word.
7. Print the final word.

# 💡 Example

Input:

Sara Omar Alterkawi

➡ Result

Words in string:

Sara

Omar

Alterkawi

## Note

- ◆ This is a manual implementation of basic tokenization.
- ◆ Punctuation remains attached to words unless it is explicitly removed.
- ◆ ``stringstream`` can provide a shorter solution for whitespace-based tokenization.



## ⊘ Warning

- ◆ ``cin >> str`` reads only one word.
- ◆ Multiple consecutive spaces require additional handling if empty tokens must be avoided.
- ◆ The final word must not be forgotten after the last delimiter is processed.





# Comparison

## ▨ `stringstream`

- ✓ Automatically extracts whitespace-separated words.
- ✓ Handles multiple whitespace characters conveniently.
- ✓ Usually produces simpler code.

## ▤ `find()` + `substr()` + `erase()`

- ✓ Gives explicit control over the tokenization process.
- ✓ Helps demonstrate how string manipulation works internally.
- ✓ Can be adapted for custom parsing logic.

# ✦✦ Summary

- ✓ Finds word boundaries using spaces.
- ✓ Extracts words using ``substr()`.`
- ✓ Removes processed sections using ``erase()`.`
- ✓ Demonstrates manual sentence tokenization.

Thank you 😊